

The Evolving World of MiniZinc

Peter J. Stuckey¹, Ralph Becket¹, Sebastian Brand¹, Mark Brown¹, Thibaut Feydy¹, Julien Fischer¹, Maria Garcia de la Banda², Kim Marriott², and Mark Wallace²

¹ National ICT Australia and the University of Melbourne, Victoria, Australia
`{pjs,rafe,sbrand,mark,tfeydy,juliensf}@csse.unimelb.edu.au`

² Monash University, Victoria, Australia
`{mbanda,marriott,mgw}@csse.monash.edu.au`

Abstract. MiniZinc arose as a response to the extended discussion at CP2006 of the need for a standard modelling language for CP. This is a challenging problem, and we believe MiniZinc makes a good attempt to handle the most obvious obstacle: there are hundreds of potential global constraints, most handled by few or no systems. We have now used MiniZinc for 2 years, and learnt a lot about its strengths and weaknesses. MiniZinc 1.0 was released in April this year. In this experience report, we discuss the features of MiniZinc that we believe are most pertinent to making a realistic standard, and what we have added and subtracted from the language to make the task of the solver developer easier. We will also talk about the toolset development for MiniZinc, and resources we have made available and plan to make available for using MiniZinc.

1 Introduction

Many constraint satisfaction and optimisation problems can be solved by CP solvers that use finite domain (FD) and linear programming (LP) techniques. There are many different solving techniques, and so there are many solvers. However, these solvers use different, incompatible modelling languages that express problems at varying levels of abstraction. This makes life difficult for modellers—if they wish to experiment with different solvers, they must learn new modelling languages and rewrite their models. A standard CP modelling language supported by multiple solvers mitigates this problem. A standard CP modelling language also makes solver benchmarking simpler, which stimulates competition between solver writers and leads to improvements in solver technology.

MiniZinc [1] was our response to the call for a standard CP modelling language. MiniZinc is high-level enough to express most CP problems easily and in a largely solver-independent way; for example, it supports sets, arrays, and user-defined predicates, some overloading, and some automatic coercions. However, MiniZinc is low-level enough that it can be mapped easily onto many solvers. For example, it is first-order, and it only supports decision variable types that are supported by most existing CP solvers: integers, floats, Booleans and sets of integers. Other MiniZinc features include: it allows separation of a model from

its data; it provides a library containing declarative definitions of many global constraints; and it also has a system of annotations which allows non-declarative information (such as search strategies) and solver-specific information (such as variable representations) to be layered on top of declarative models.

Crucially for a standard to become realised it must be as simple as possible for solver writers to support the standard. This is the fundamental design goal of MiniZinc. MiniZinc models are translated to FlatZinc, a low-level solver input language that is the target language for MiniZinc. FlatZinc is designed to be easy to translate into the form required by a CP solver. We provide many features in order to make this translation specializable for a particular solver: including a modifiable library of global constraint definitions, facilities to rewrite the base FlatZinc constraints, and annotations to transport information from the model to the solver. In this paper we detail how we have modified the original design of MiniZinc and FlatZinc in order to simplify the life of solver writers and allow them more flexibility in supporting MiniZinc.

MiniZinc is currently supported by constraint programming solvers including Eclipse [2], Gecode [3] and Sicstus Prolog [4]. The G12 project has a number of constraint programming solvers that can execute MiniZinc: the G12 FD solver, the hybrid G12 FD/SAT lazy clause generation solver [5], and a hybrid finite domain/linear programming solver. Since MiniZinc is a solver independent language it is also possible to support other kinds of solvers. We think it is important to compare CP solvers against SAT and MIP technology on the same problems. To this end, in G12, we have tools for MiniZinc that translate FlatZinc into CPLEX LP format, and also a tool, *fzntini* [6], that works by translating to SAT.

We have used MiniZinc in a number of offerings of constraint programming and optimization subjects. Our experience is that students find it certainly easier to learn than the CLP based modelling approaches that we previously taught. But the courses highlight the need for good tool support, and in particular better generic visualization and debugging facilities. We plan to make MiniZinc courseware available as we polish our offerings.

There are other potential candidates for standard CP modelling languages such as OPL [7] (very similar to MiniZinc but not extensible), XCSP [8] (a problem definition language rather than modelling language), Essence (or Essence') [9] (a high level modelling language allowing choice of rewriting), or perhaps Comet [10] (a procedural CP language with good modelling facilities). We believe the chief advantage of MiniZinc over all of these is the ease in which a solver developer can support the language.

2 MiniZinc by Example

Considering the workshop audience we assume that most readers are familiar with CP modelling, hence we introduce MiniZinc by example.

Consider the problem of minimizing the maximum number of open stacks. A factory manufactures a number of different products in batches, i.e., all copies of

a given product need to be finished before a different product is manufactured, so there are never two batches of the same product. Each customer of the factory places an order requiring one or more different products. Once one product in a customer's order starts being manufactured, a stack is opened for that customer to store all products in the order. Once all the products for a particular customer have been manufactured, the order can be sent and the stack is freed for use by another order. The aim is to determine the sequence in which products should be manufactured to minimize the maximum number of open stacks, i.e., the maximum number of customers whose orders are simultaneously active.

For c customers and p products we have a $c \times p$ array `orders` of 0..1 which shows for each customer what products they order. For example, the MiniZinc data file `os.dzn` for a small problem may look like:

```
c = 5;
p = 9;
orders = [| 1, 0, 1, 0, 1, 0, 1, 0, 0
           | 1, 0, 0, 1, 0, 0, 1, 0, 0
           | 0, 1, 1, 0, 1, 1, 0, 1, 0
           | 0, 1, 0, 1, 0, 0, 0, 0, 1
           | 0, 0, 0, 0, 0, 1, 0, 0, 1
           |];
```

where e.g. customer 1 orders products $\{1,3,5,7\}$. Notice the notation for two-dimensional arrays, that is a recent addition to MiniZinc.

A MiniZinc model file `os.mzn` for open stacks is the following:

```
% Open Stacks problem
include "all_different.mzn";

%-- Parameters -----
int: c; % number of customers
int: p; % number of products
set of int: Custs = 1..c;
set of int: Prods = 1..p;
array[Custs,Prods] of 0..1: orders; % products for each customer
array[Custs] of int: norders =
    [ sum(j in Prods)(orders[i,j]) | i in Custs ];
    %% total orders of each customer

%-- Decision variables -----
array[Prods] of var Prods: s :: is_output ; % schedule of products

%-- Auxiliary variables -----
array[Custs, 0..p] of var 0..p: o ; % # orders filled after time t

%-- Constraints -----
```

```

constraint all_different(s); % each product scheduled once
constraint
    forall (i in Custs)(o[i,0] = 0) % no orders at time 0
    /\
    forall(t in 1..p, i in Custs)(
        o[i,t] = o[i,t-1] + orders[i,s[t]] );

%-- Solving objective -----%

solve :: int_search(s, input_order, indomain, complete)
    minimize
        max(j in Prods)(
            sum(i in Custs)(
                bool2int( o[i,j-1] < norders[i] /\ o[i,j] > 0 )));

```

Comments are anything on a line after %. The `include` item includes a definition for the global constraint `all_different` used in the model. This may be a native global or defined by decomposition (see Section 3).

The *fixed* parameters for the model are defined with their type. If no definition is given it is assumed to come from the data file. Note that we can compute fixed parameters in terms of existing parameters, for example `Custs` and `norders`.

The decision variables are declared using a `var` type. The annotation `:: is_output` declares that we want this value printed out by the program at the end. Here the decisions represent the sequence of the products produced. We may well introduce auxiliary variables as well, like `o`, which represents how many orders for each customer are filled at each time.

Constraints in the model are items that begin with the `constraint` keyword. The constraints ensure that each product is scheduled once (using `all_different`), and determine the meaning of the `o` variables. No orders are filled initially, and at time t the number of orders filled (products built) for customer i are $o[i, t - 1]$ plus 1 if customer i orders the product $s[t]$ scheduled at time t .

The item starting with the keyword `solve` determines what kind of problem is being solved. The goal here is to minimize the objective function which is the maximum of the open stacks at each time. It counts open stacks for customers which have at least 1 but not all products built. The `bool2int` coerces the Boolean result of the test into a 0..1 integer. The `solve` item is annotated with a simple builtin search annotation. Since the model is meant to be independent of solving technique the search is encoded in annotations. This search annotation means fix all the variables `s` in the order entered, trying their values from lowest to highest, and do a complete search.

MiniZinc translation to FlatZinc is invoked by

```
mzn2fzn os.mzn --data os.dzn
```

which creates a FlatZinc file `os.fzn`. This can be executed using a FlatZinc solver by

```
flatzinc os.fzn
```

When using the G12 FD solver one can also invoke the minizinc one pass interpreter as follows:

```
minizinc os.mzn --data os.dzn
```

For the above example both these output

```
s = array1d(1..9, [1, 3, 5, 7, 2, 4, 6, 8, 9]);
```

just showing the values of the decision (output) variables.

MiniZinc also supports complex output expressions. For example we could add the output item

```
output [ if (0 < fix(o[i,j]) /\ fix(o[i,j-1]) < norders[i]) then
          if orders[i,fix(s[j])] > 0 then "X " else "- "
        endif else "." endif
        ++ if j = p then "\n" else "" endif |
        i in Custs, j in Prods ] ++
[ show(s[j]) ++ " " | j in Prods ] ++ ["\n"] ;
```

to this model, in which case the output (from minizinc) is:

```
X X X X . . . . .
X - - X - X . . .
. X X - X - X X .
. . . . X X - - X
. . . . . X - X
1 3 5 7 2 4 6 8 9
```

which illustrates the sequence of products and for each customer at each time whether it has no open stack (.) or is open with a product scheduled (X) or no product scheduled (-).

3 Global Constraints

A primary goal for MiniZinc was to support as broad a range of solvers as possible without restricting the allowable set of global constraints. To this end global constraints are always provided first as a predicate defined by decomposition. Solver writers can override this definition if their solver supports the global constraint directly or if they wish to use an alternative decomposition.

MiniZinc provides reflection functions like `ub`, `lb`, `dom` (returning the declared upper, lower bounds and domain of variables or arrays of variables) and `index_set` (returning the index set of an array) to make the construction of decompositions reasonably straightforward. For example here is the cumulative sum decomposition [11] of the `sequence` constraint

```

%-----%
% Requires that in each subsequence
%       'x[i], ..., x[i + 1 - 1]'
% the sum of the variables is between 'mn' and 'mx'.
%-----%
predicate
sequence(array[int] of var int: x, int: l, int: mn, int: mx) =
    let { int: iu = max(index_set(x)),
          int: il = min(index_set(x)),
          int: n = iu - il + 1,
          int: maxs = n * ub(x),
          int: mins = n * lb(x),
          array[il-1..iu] of var mins..maxs: s } in
    s[i1-1] == 0 /\
    forall(i in il..iu)(s[i] == x[i] + s[i-1]) /\
    forall(i in il-1..iu-1)(
        mn <= s[i+1] - s[i] /\ s[i+1] - s[i] <= mx);

```

Note how we use `index_set` to find the bounds on the index set of the sequence passed in, and used the declared bounds on all `x` variables to determine the range of the introduced sum variables `s`.

Since MiniZinc 1.0 global constraint definitions each reside in their own separate model file. To use the global it must be included in the model, e.g.

```
include "sequence.mzn";
```

One can still include all globals by

```
include "globals.mzn";
```

As mentioned, solver writers can override the default definition of a global constraint. For example supposing a solver supports the `global_cardinality` (gcc) constraint, they might replace the default definition of `all_different`

```

predicate all_different(array[int] of var int: xs) =
    forall(i,j in index_set(xs) where i < j)(xs[i] != xs[j]);

```

by a version using `global_cardinality`

```

predicate all_different(array[int] of var int: xs) =
    let { set of int: D = dom(xs),
          array[D] of var 0..1: cards
        } in global_cardinality(xs, cards);

```

Arbitrary new global constraints can be supported by a solver. They are signalled by just giving a predicate declaration with no body. For example a solver with a native `all_different` simply uses the declaration

```
predicate all_different(array[int] of var int: xs);
```

We have refined the structure of the MiniZinc library so that solver developers can add new directories containing solver-specific definitions of global constraints that their solver supports directly. The default decompositions are used to provide the definitions of any remaining global constraints.

The MiniZinc 1.0 release includes around 50 of the 150+ global constraints defined in the global constraints catalog [12]. We try to make the signatures of the global constraints match those in the catalog, but have to change them when the catalog version uses types not supported by MiniZinc/FlatZinc.

MiniZinc always had assertion items for checking that parameters given to a model were sensible. We extended the assertion syntax to allow assertions to be placed on any expression. This is particularly useful for checking conditions on the arguments of global constraints.

We have found the facilities for globals to be quite expressive. For example adding `table` constraints for integers to MiniZinc simply required adding the definition below (which effectively maps them to `element` constraints):

```
predicate
  table(array[int] of var int: x, array[int, int] of int: t) =
    assert (index_set_2of2(t) == index_set(x),
      "The second dimension of the table must equal " ++
      "the number of variables in the first argument",
    let { int: lt = min(index_set_1of2(t)),
        int: ut = max(index_set_1of2(t)),
        var lt..ut: i }
    in
      forall(j in index_set(x)) (
        t[i,j] = x[j]
      )
    );
```

Of course solvers that natively support `table` only need to declare the predicate without definition.

4 Annotations

Solver writers can communicate to their solver using *annotations*, an extensible markup language for MiniZinc. Annotations in MiniZinc are prepended after a `::` operator. They can be attached to variable declarations, constraints, predicates and to a model's solve item. Annotations provide a general method to add information to a model which is not part of the solver-independent model. For example we can pass information to a global constraint to control its behaviour. An annotated constraint

```
constraint cumulative(s, d, r, limit) :: edge_finding;
```

in the model will be passed through to FlatZinc unchanged (assuming there is no decomposition for `cumulative`), and the FlatZinc solver can interpret the annotation `edge_finding` as how the cumulative global should be executed.

Annotations are now declared with their type so that these types can be checked. For example, the simple edge finding annotation is declared as

```
annotation edge_finding;
```

A limited discrepancy search annotation (which only examines the first k discrepancies) is declared as:

```
annotation lds(int:k);
```

We introduced the type `ann`, principally to allow the use of nested annotations. So for example an annotation for sequencing different searches may be declared as

```
annotation seq_search(array[int] of ann);
```

which allows a search annotation such as

```
seq_search([int_search(xs, smallest, indomain_min, complete),
           first_fail]);
```

to be defined, which says first label the `xs` variables by labeling the variable with the smallest possible value to that value, and when they are all fixed, then label the rest of variables using first fail.

One of the consequences of introducing the annotation type is that we no longer use string constants to express part of complex annotations. Hence "`lds(3)`" in previous versions of MiniZinc is now `lds(3)`.

Annotation variables are another recently introduced feature. They act as placeholders for annotations and allow the actual annotation value to be specified separately from the model.

For example, in the following fragment we use the annotation variable `S` on the solve item as a place holder for the search annotation:

```
ann: S;
solve :: S satisfy;
```

We can then provide the value of `S` separately in a data file:

```
S = int_search(xs, smallest, indomain_min, complete);
```

This allows the modeller to experiment with different search strategies without having to modify the model.

A solver writer can define the set of annotations supported by their solver. This does not stop a modeller introducing new annotations in their model. All annotations are allowed to be ignored by solvers.

5 Complex Expressions

MiniZinc allows arbitrarily complex constraint expressions, which are translated to flat constraint conjunctions by decomposition. For example

```
constraint (x1 - x2)*(x1 - x2) + (y1 - y2)*(y1 - y2) >= d;
```

where the variables range from 0..10 is flattened to (something equivalent to) the FlatZinc

```
var -10..10: t1 :: var_is_introduced :: is_defined_var;
var -10..10: t2 :: var_is_introduced :: is_defined_var;
var -100..100: t3 :: var_is_introduced :: is_defined_var;
var -100..100: t4 :: var_is_introduced :: is_defined_var;
constraint int_lin_eq([1,-1,1], [t1,x1,x2], 0) :: defines_var(t1);
constraint int_lin_eq([1,-1,1], [t2,y1,y2], 0) :: defines_var(t2);
constraint int_times(t1, t1, t3) :: defines_var(t3);
constraint int_times(t2, t2, t4) :: defines_var(t4);
constraint int_lin_ge([1,1,-1], [t3,t4,d], 0);
```

The flattening process performs (simple) bounds analysis to avoid introducing unbounded integer variables if possible, and common subexpression elimination.

The annotation pair `is_defined_var/defines_var` allows a solver to reconstruct the original term form from the flattened form if so desired. For example interval solvers usually prefer to handle the original constraint as one expression than treat the decomposed version.

The annotation `var_is_introduced` is useful for solvers to determine how their default search procedure should handle the variable.

MiniZinc is careful to preserve the *relational semantics* (see [13]) of the model in this flattening of nested expressions, that is where partial functions are considered to cause the nearest enclosing Boolean context to be false when the result of the partial function is not defined. The most common partial function in models are arrays. So for example

```
var 1..10: i;
array[1..5] of int: a = [-1,3,0,-2,4];
constraint a[i] != 0 \ / i > 8;
```

does not force that `i` takes a value in 1..5 because of its use as an index of the array `a`³, nor aborts if `i` takes the value 6.

6 FlatZinc

The principal tool support for MiniZinc is `mzn2fzn` which translates MiniZinc to FlatZinc.

³ Forcing `i` to be in 1..5 is correct if we consider the *strict semantics* [13] of partial functions.

In order to make FlatZinc files more standalone, the translator now includes predicate type declarations for all global constraints that appear in the FlatZinc file. This allows processing of the FlatZinc files by other tools than the intended solver. This facility can be turned off for compatibility with earlier versions.

In order to allow specialization for particular solvers authors can write predicate definitions for FlatZinc builtins which are then used to rewrite the output. For example suppose a solver does not support the FlatZinc element constraint for integers, which has FlatZinc type

```
array_int_element(var int, array[int] of int, var int);
```

They can add a definition like that below to their model (which is actually a good translation for SAT style solvers).

```
predicate
  array_int_element(var int:i, array[int] of int:a, var int:x) =
    forall(j in index_set(a))( i = j -> x = a[j] );
```

Then the resulting models from `mzn2fzn` will not include `array_int_element`.

We can also use this mechanism to reduce the burden on solver writers in writing equivalent FlatZinc constraints. For example the definition

```
predicate int_le(var int:x, var int:y) = int_ge(y,x);
```

means that the solver will never see an `int_le` constraint (it will be replaced by `int_ge`). We plan to deprecate a number of FlatZinc constraints in the future in this way.

One danger that this mechanism adds is the possibility of looping translation behaviour. For predicates in MiniZinc this is explicitly checked but here there is a dependency from MiniZinc constructs to FlatZinc which is not captured. For example

```
predicate int_le(var int:x, var int:y) = x <= y;
```

will cause infinite looping in the present system, since `x <= y` translates to `int_le(x,y)` and then to `x <= y` again.

Since MiniZinc 1.0 we also provide tools to translate FlatZinc to an XML form, and back again. This is to aid solver developers who prefer to use XML based parsing approaches, although one should note that the XML form is much larger than the textual FlatZinc.

FlatZinc is relatively compact. Indeed it has been used to dump out internal states of a constraint programming solver into textual form, in order to use disk storage (useful for very large long running searches). It was found that the FlatZinc representation was about the same size as the internal solver representation, meaning that it is quite concise.

7 Output

MiniZinc allows the construction of quite complex output from the results of a model, using the `fix` function which extracts the fixed value from a variable (and aborts if it is not fixed). Consider the output for an n -queens model

```
output [ if fix(q[i]) = j then "Q " else ". " endif ++
        if j = n then "\n" else "" endif
      | i, j in 1..n
    ];
```

which for $n = 5$ might output something like

```
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .
```

In order to simplify the implementation of FlatZinc solvers, output items are no longer included in FlatZinc. Solvers are thus not required to handle complex output. The only output in FlatZinc is the outputting of FlatZinc variables possibly with a coercion to the correct MiniZinc type. The `mzn2fzn` translation creates output annotations for variables used in the output item, for example the output item above will create FlatZinc for which includes the definition

```
array[1..5] of var 1..5: q :: output_array([1..5]);
```

which requests the solver to output the `q` values on completion, with an array coercion to ensure the right interpretation. The solver only outputs

```
q = array1d(1..5, [1, 3, 5, 2, 4]);
```

This can then be used as input to the MiniZinc interpreter `minizinc` to recover the pretty output above.

Note that the coercion is required to handle the restrictions on arrays in FlatZinc, which must be one dimensional and starting from 1. For example the MiniZinc array

```
array[2..8, -4..0] of var -1..1: x :: is_output;
```

becomes in FlatZinc

```
array[1..35] of var -1..1: x :: output_array([2..8, -4..0]);
```

From experience in setting up testing scripts we are now much more prescriptive on the output requirements of FlatZinc. This makes it is easy to compare different solvers on the same model. Each successful solution is followed by a line “-----”. When there are no more solutions there is a line “=====”. For example for 5-queens where the first queen appears before the last, all solutions are output as:

```

q = array1d(1..5, [1, 3, 5, 2, 4]);
-----
q = array1d(1..5, [1, 4, 2, 5, 3]);
-----
q = array1d(1..5, [2, 4, 1, 3, 5]);
-----
q = array1d(1..5, [2, 5, 3, 1, 4]);
-----
q = array1d(1..5, [3, 1, 4, 2, 5]);
-----
=====

```

This allows simple solution counting and post processing. We also allow optimization problems to output increasingly better solutions so that we can cut the computation at any time and know the best solution found so far.

8 Ongoing Work

We are continually improving the tool set for MiniZinc, in order to make it easier to use and support. Below are a number of our current endeavours.

8.1 MiniZinc-to-MiniZinc Transformations

The currently predominant use of MiniZinc is as an input language to the `mzn2fzn` program which translates it to FlatZinc. Rather than extending `mzn2fzn` with additional functionality that may only be weakly linked to the FlatZinc translation, we are developing separate MiniZinc-to-MiniZinc transformations. The tool for this is the purpose-built model transformation system Cadmium [14].

Transformations are typically formulated in a symbolic way, and are thus naturally applied to the model independently of its instance data. This can save much transformation work if the expression being rewritten occurs in a comprehension that generates multiple instances of it, such as a `forall` loop. For example, the disjunction in the model fragment

```

array[p..q] of var 1..u: x;
constraint forall(i in p+1..q-1)(
    x[i] >= x[i-1]  \/  x[i] >= x[i+1] );

```

needs to be linearized if a linear solver is to be used. One possibility is to unroll the `forall` during the conversion to FlatZinc and then linearize the resulting FlatZinc model (see Section 6). However, linearization can also take place without flattening. The following fragment has the desired form:

```

array[p..q] of var 1..u: x;
array[p+1..q-1] of var 0..1: b;

```

```
constraint forall(i in p+1..q-1)(
    0 <= (u-1)*b[i] + x[i] - x[i-1]
    /\ 1-u <= (1-u)*b[i] + x[i] - x[i+1] );
```

It was produced by our data-independent MiniZinc linearization. Several independent subtransformations are joined for this result: Big-M linearization of disjunctions, bounds inference evaluating the `lb/ub` reflection functions to calculate the Big-Ms, local-variable lifting to move the declaration of the indicator variable `b` from inside the `forall` (where it arises) to the top-level as an array, and generic expression simplification and normalization.

We greatly benefit from the support for predicates in MiniZinc. The linearized forms of more complex expressions (such as variable index array lookups) and specialised linear versions of global constraints (such as `all_different`) are all defined in a regular MiniZinc library. The transformation inserts calls to them as appropriate. Being based on user-definable predicates makes the linearization easily configurable.

We plan to make Cadmium and various MiniZinc-to-MiniZinc transformations freely available to the academic community in the medium term.

8.2 Interactive Development Environment

We are in the process of developing an Eclipse based IDE for MiniZinc, which provides a runtime environment as well as contextual editing and interaction with error messages. We hope to release this before the workshop. The IDE also provides a graphical output language to define visualizations of the results of models, which we plan to extend to visualize solving as well.

8.3 Challenge

The MiniZinc Challenge [15] provides a comparison of different solvers supporting MiniZinc on a set of problems. Each of the models has a defined search strategy which must be followed in the “fixed search” category but can be ignored in the “free search” category. The MiniZinc challenge like the CSP solving competition [16] allows us to compare solvers on the same problems. In contrast to the CSP solving competition it compares solvers using the same search strategy, and concentrates on optimization, rather than satisfaction, problems.

It is unlikely that there will ever be one best solver, so the importance of the challenge is to compare systems as closely as systems on a range of different style of problems. The purse points system rewards solvers that are the best at a certain class of problems. We also think that the participation of non-standard-CP MiniZinc/FlatZinc solvers (e.g. SAT, ILP, hybrids) in the challenge makes it clearer where the strengths and weaknesses of traditional CP technology lie and so help drive its development.

8.4 Wiki

The MiniZinc release includes a significant number of models and data files, as well as expected outputs, and indeed conformance testing scripts to test that a solver produces the expected results. This is very useful for building a MiniZinc interface to a solver, and indeed testing a solver.

But there is a huge wealth of modelling experience out there. A great benefit of a standard modelling language is to have a large library of problems available with data files, best known solutions, etc., so that comparisons are easy to make. We have just made available a public wiki www.g12.csse.unimelb.edu.au/wiki/ of problems and models written in MiniZinc in order to help further experimentation in the field. The wiki will include problem based sections where alternate models of the same problem are given, as well as global sections where alternate decompositions of global constraints are given. We also will include testing various aspects of MiniZinc. The wiki also gives a place for modellers and developers to find all the tools available for using MiniZinc.

By making a wiki we hope that the constraint programming community will become involved in building a living repository of constraint programming experience.

9 Conclusion and Future Work

Our main goal with MiniZinc is to define a language that is not too big but expressive enough to succinctly capture most CP problems. Our hope is that by providing simple-to-use tools for manipulating MiniZinc and FlatZinc it will be easy for solver writers to support this proposed standard, which will lead to benefits for both solver users and solver writers.

To make MiniZinc a complete standard there are still many questions to address, for example: how do we write a suitably expressive search language that is easy enough for (at least constraint programming) solvers to support; how do we make the integration of MiniZinc models into applications seamless and as easy as possible; how do we create effective debugging interfaces so that MiniZinc models can be debugged in a manner somewhat independent of the actual solver being used.

Acknowledgements

Thanks to Christian Schulte, Mikael Lagerkvist, Joachim Schimpf, Guido Tack, and Radoslaw Szymanek for helpful discussions about MiniZinc and FlatZinc.

References

1. Nethercote, N., Stuckey, P., Becket, R., Brand, S., Duck, G., Tack, G.: Minizinc: Towards a standard CP modelling language. In Bessiere, C., ed.: *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*. Volume 4741 of LNCS., Springer-Verlag (2007) 529–543

2. Apt, K., Wallace, M.: Constraint Logic programming Using ECLiPSe. Cambridge University Press (2007)
3. Schulte, C., Lagerkvist, M., Tack, G.: Gecode. <http://www.gecode.org/>
4. : Sicstus prolog. <http://www.sics.se/sicstus/>
5. Feydy, T., Stuckey, P.: Lazy clause generation reengineered. In Gent, I., ed.: Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming. Volume 5732 of LNCS., Springer-Verlag (2009) to appear
6. Huang, J.: Universal Booleanization of constraint models. In Stuckey, P., ed.: 14th Int. Conf. on Principles and Practice of Constraint Programming (CP'08). Volume 5202 of LNCS., Springer (2008)
7. Van Hentenryck, P.: The OPL Optimization Programming Language. MIT Press (1999)
8. : Xcsp 2.1. <http://www.cril.univ-artois.fr/CPAI08/XCSP2.1.pdf>
9. Frisch, A.M., Harvey, W., Jefferson, C., Hernández, B.M., Miguel, I.: Essence : A constraint language for specifying combinatorial problems. *Constraints* **13**(3) (2008) 268–306
10. Van Hentenryck, P., Michel, L.: Constraint-Based Local Search. MIT Press (2005)
11. Brand, S., Narodytska, N., Quimper, C.G., Stuckey, P., Walsh, T.: Encodings of the sequence constraint. In Bessiere, C., ed.: Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming. Volume 4741 of LNCS., Springer-Verlag (2007) 210–224
12. : Global constraint catalog. <http://www.emn.fr/x-info/sdemasse/gccat/>
13. Frisch, A., Stuckey, P.: The proper treatment of undefinedness in constraint languages. In Gent, I., ed.: Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming. Volume 5732 of LNCS., Springer-Verlag (2009) to appear
14. Duck, G., Stuckey, P., Brand, S.: ACD term rewriting. In: Proceedings of ICLP 2006. Number 4079 in LNCS, Springer-Verlag (August 2006) 117–131
15. : Minizinc challenge. <http://www.g12.csse.unimelb.edu.au/minizinc/challenge2009/challenge.html>
16. : Csp solver competition. <http://www.cril.univ-artois.fr/CPAI09/>